

100 Essential Easy LeetCode Problems for Interviews

Mastering easy LeetCode problems builds a solid foundation in core DSA concepts – arrays, strings, hashing, basic math, etc. – which interviewers often test ¹ ². In fact, experienced interview coaches emphasize **easy** problems as primary interview fodder: you're likely to face an easy problem under time pressure, so focus on these basics ² ³. We compiled **100** key easy problems (with links, topics, and hints), covering all major categories (arrays, strings, linked lists, trees, math/logic, search/sort, stacks/queues, bit tricks, etc.). This list is drawn from popular sources and interview recommendations ⁴ ³. We also include study tips (e.g. practice order, revision) at the end.

Arrays & Hashing

Recommended array tasks (from Top-50 lists ⁴ and community guides ³) include classics like **Two Sum**, **Best Time to Buy/Sell Stock**, **Contains Duplicate**, **Rotate Array**, etc. These problems use hash tables, two-pointers, or simple scans to solve common patterns. Practice them early to build speed with array manipulations and hash maps.

- **Two Sum** – Tags: Array, Hash Table. Given `nums` and a target, find two indices whose values sum to target. *Hint:* Use a hash map to record seen values and look up the complement in one pass.
- **Two Sum II - Input Array Is Sorted** – Tags: Array, Two Pointers. Like Two Sum but array is sorted. *Hint:* Use two pointers (one at start, one at end) and move inward based on the sum.
- **Best Time to Buy and Sell Stock** – Tags: Array, Greedy. Given daily prices, find max profit from one buy-sell. *Hint:* Track the minimum price seen so far and update max profit by comparing current price minus min so far.
- **Contains Duplicate** – Tags: Array, Hash Table. Check if any value appears twice in the array. *Hint:* Use a hash set: if a number is already in the set, return true; otherwise add it.
- **Intersection of Two Arrays II** – Tags: Array, Hash Table. Find intersection (with multiplicity) of two arrays. *Hint:* Use a hash map to count elements of one array, then for the other array check and decrement counts.
- **Single Number** – Tags: Array, Bit Manipulation. In an array where every element appears twice except one, find the unique one. *Hint:* XOR all numbers together; duplicates cancel out, leaving the unique number.
- **Majority Element** – Tags: Array, Hash Table. Find the element that appears more than $\lfloor n/2 \rfloor$ times. *Hint:* Use a hash map count or Boyer-Moore Voting algorithm (track a candidate and count).
- **Plus One** – Tags: Array, Math. Given an array-form integer (e.g. [1,2,3]), add one to the number. *Hint:* Add one from the least significant digit, handling carry; if carry remains, prepend 1.
- **Move Zeroes** – Tags: Array, Two Pointers. Move all zeros in the array to the end while maintaining order of non-zeros. *Hint:* Use two pointers: one to traverse the array, and one to keep track of the position to place the next non-zero.

- **Remove Duplicates from Sorted Array** – Tags: Array, Two Pointers. Given a sorted array, remove duplicates in-place and return new length. *Hint:* Use a slow-runner pointer for the position of next unique element and a fast-runner scanning the array.
- **Search Insert Position** – Tags: Array, Binary Search. Find the index where a target should be inserted in a sorted array. *Hint:* Use binary search to locate the insertion point.
- **Rotate Array** – Tags: Array. Rotate the array to the right by k steps. *Hint:* Reverse parts of the array: reverse all, then reverse the first k, then reverse the rest.
- **Sort Colors** – Tags: Array, Two Pointers. Given array of 0s,1s,2s (colors), sort in-place. *Hint:* Use the Dutch National Flag algorithm with three pointers.
- **3Sum** – Tags: Array, Two Pointers. (Medium difficulty, but foundational.) Find triplets summing to zero. *Hint:* Sort the array and use two pointers for the inner loop.



Strings

Key string-manipulation problems often appear in interviews ⁵. Work on problems like **Valid Palindrome**, **Implement strStr()**, **Longest Common Prefix**, etc. These reinforce looping and hash/map usage on characters, or two-pointer sliding window.

- **Valid Palindrome** – Tags: String, Two Pointers. Check if a string reads the same forwards/backwards (ignoring non-alphanumeric and case). *Hint:* Use two pointers (start/end), skip non-alphanumerics, compare characters.
- **Implement strStr()** – Tags: String, Two Pointers. Return the index of the first occurrence of a needle in a haystack (or -1). *Hint:* A straightforward nested loop (or Knuth-Morris-Pratt for bonus).
- **Longest Common Prefix** – Tags: String. Find longest common prefix string among an array of strings. *Hint:* Compare characters column-wise, or sort the array and compare only first and last string.
- **First Unique Character in a String** – Tags: String, Hash Table. Return index of first non-repeating char. *Hint:* Count frequency with a hash map, then scan for the first char with count 1.
- **Reverse String** – Tags: String. Reverse an array of characters in-place. *Hint:* Use two pointers swapping from ends.
- **Reverse Integer** – Tags: Math, String. Reverse digits of an integer. *Hint:* Pop digits one-by-one (or convert to string and reverse); watch out for overflow.
- **Valid Parentheses** – Tags: String, Stack. Check if parentheses are well-formed. *Hint:* Use a stack: push open brackets, and on closing bracket check if it matches the top.
- **Count and Say** – Tags: String. Generate the nth count-and-say sequence term. *Hint:* Build each term by reading off digits of the previous term (string manipulation).
- **Add Binary** – Tags: String, Math. Add two binary strings. *Hint:* Simulate binary addition from LSB to MSB with carry.
- **Ransom Note** – Tags: String, Hash Table. Given two strings (ransom and magazine), check if ransom can be constructed from magazine. *Hint:* Use a hash map to count magazine characters and decrement as you “use” them for ransom.
- **Implement strStr() – Case Sensitive** – Tags: String. (Same as strStr above, to emphasize substring search technique.) *Hint:* [See Implement strStr() hint.]*



Linked Lists

Basic linked list operations are common interview topics. Typical easy problems include reversing a list, merging sorted lists, detecting/removing cycles, etc. A recommended list of 50 easy problems includes Reverse Linked List, Merge Two Sorted Lists, Cycle detection, etc ⁶.

- [Reverse Linked List](#) – *Tags:* Linked List. Reverse a singly linked list. *Hint:* Use three pointers (prev, curr, next) to iteratively reverse links.
- [Merge Two Sorted Lists](#) – *Tags:* Linked List, Recursion. Merge two sorted linked lists into one sorted list. *Hint:* Use a dummy head and iterate through both lists, picking smaller nodes (or use recursion).
- [Remove Linked List Elements](#) – *Tags:* Linked List. Remove all nodes with a target value. *Hint:* Use a dummy head to handle head removals; iterate and skip nodes with target value.
- [Palindrome Linked List](#) – *Tags:* Linked List, Two Pointers. Check if a list is a palindrome. *Hint:* Use fast/slow pointers to find mid, reverse second half, and compare halves.
- [Linked List Cycle](#) – *Tags:* Linked List, Two Pointers. Detect if a cycle exists in a list. *Hint:* Use the tortoise-and-hare (slow/fast pointers) algorithm.
- [Intersection of Two Linked Lists](#) – *Tags:* Linked List, Two Pointers. Find node where two singly linked lists intersect. *Hint:* Use two pointers that traverse each list; when one ends, start at the other list. They will meet at intersection or both at null.
- [Delete Node in a Linked List](#) – *Tags:* Linked List. Given node (not head) to delete, remove it. *Hint:* Copy next node's value into current and skip the next node.
- [Remove Duplicates from Sorted List](#) – *Tags:* Linked List. Remove duplicate values from a sorted list (each value appears once). *Hint:* Iterate with a pointer; if `node.val == node.next.val`, skip the next node.
- [Odd Even Linked List](#) – *Tags:* Linked List. Group all odd-index nodes together followed by even-index nodes. *Hint:* Maintain two lists (odd and even) and stitch them.
- [Add Two Numbers](#) – *Tags:* Linked List. Add two numbers represented by linked lists (digits in reverse order). *Hint:* Use two pointers and carry to add digits; build the resulting list.



Trees & Binary Search Trees

Basic binary tree problems (max depth, symmetric check, tree inversion, etc.) are essential interview material. A typical easy set includes *Maximum Depth*, *Symmetric Tree*, *Invert Binary Tree*, *Path Sum*, *Same Tree*, etc. ⁷. For binary search trees, include *Validate BST*, *Convert Sorted Array to BST*, and simple traversals.

- [Maximum Depth of Binary Tree](#) – *Tags:* Tree, DFS. Find max depth. *Hint:* Use recursion: `depth = 1 + max(leftDepth, rightDepth)`.
- [Symmetric Tree](#) – *Tags:* Tree. Check if tree is mirror symmetric. *Hint:* Use a helper to compare left-subtree of one side with right-subtree of the other side (two-pointer style).
- [Invert Binary Tree](#) – *Tags:* Tree, DFS/BFS. Invert (mirror) the binary tree. *Hint:* Swap left/right children recursively or with a BFS queue.
- [Path Sum](#) – *Tags:* Tree, DFS. Check if a root-to-leaf path sums to target. *Hint:* Recursively subtract node values from target and check leaf.
- [Same Tree](#) – *Tags:* Tree, DFS. Check if two trees are identical. *Hint:* Recursively compare node values and subtrees.
- [Validate Binary Search Tree](#) – *Tags:* Tree, DFS. Check if tree is a valid BST. *Hint:* Inorder traversal should yield sorted values, or propagate min/max bounds in recursion.

- [Binary Tree Inorder Traversal](#) – Tags: Tree, DFS. Return inorder traversal. *Hint:* Use recursion (left-root-right) or iterative with a stack.
- [Binary Tree Preorder Traversal](#) – Tags: Tree, DFS. Return preorder (root-left-right). *Hint:* Use recursion or stack.
- [Binary Tree Postorder Traversal](#) – Tags: Tree, DFS. Return postorder (left-right-root). *Hint:* Use recursion or modified stack approach.
- [Convert Sorted Array to BST](#) – Tags: Tree, Recursion. Build a height-balanced BST from sorted array. *Hint:* Recursively pick mid element as root, build left/right subtrees.
- [Balanced Binary Tree](#) – Tags: Tree. Check if tree height difference ≤ 1 at every node. *Hint:* Recursively compute heights; if any subtree is unbalanced, propagate failure.
- [Diameter of Binary Tree](#) – Tags: Tree, DFS. Longest path length between any two nodes. *Hint:* For each node, the diameter can be $\text{leftHeight} + \text{rightHeight}$; compute with post-order DFS, track max.

Dynamic Programming / Math & Logic

A few simple DP and math problems are high-yield for interviews. Examples include **Climbing Stairs**, **House Robber**, **Maximum Subarray** (DP version), and math puzzles like **Fizz Buzz**, **Count Bits**, etc. ⁸ ⁹. These reinforce understanding of recursion/DP and arithmetic logic.

- [Climbing Stairs](#) – Tags: Math, DP. Count ways to climb n stairs (1 or 2 steps). *Hint:* Fibonacci relation: $\text{ways}[n] = \text{ways}[n-1] + \text{ways}[n-2]$.
- [House Robber](#) – Tags: Array, DP. Max sum of non-adjacent numbers in an array. *Hint:* DP recurrence: $\text{dp}[i] = \max(\text{dp}[i-1], \text{dp}[i-2] + \text{nums}[i])$.
- [Maximum Subarray](#) – Tags: Array, DP. Find max sum of a contiguous subarray. *Hint:* Kadane's algorithm: keep a running sum (reset to 0 if negative) and track max.
- [Min Cost Climbing Stairs](#) – Tags: DP. Given cost array, find min cost to reach top. *Hint:* dp recurrence: $\text{dp}[i] = \text{cost}[i] + \min(\text{dp}[i-1], \text{dp}[i-2])$.
- [Fizz Buzz](#) – Tags: Math, Simulation. Classic fizz buzz output. *Hint:* Loop from 1 to n , check divisibility by 3 and/or 5.
- [Count and Say](#) – Tags: String, Math. (Listed under strings above; it is also a simple sequence problem.)
- [Happy Number](#) – Tags: Math, Hash Set. Check if sum-of-squares-of-digits process reaches 1 or loops. *Hint:* Use a set to detect cycles of sums.
- [Power of Two](#) – Tags: Math, Bit Manipulation. Check if n is a power of two. *Hint:* Use the bit trick `$n \ \& \ (n-1) == 0$` (and $n > 0$).
- [Power of Three](#) – Tags: Math. Check if n is a power of three. *Hint:* Repeatedly divide by 3.
- [Factorial Trailing Zeroes](#) – Tags: Math. Count trailing zeros in $n!$. *Hint:* Count factors of 5: $\text{sum} \lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \dots$
- [Excel Sheet Column Number](#) – Tags: Math. Convert column title (e.g. "AB") to number. *Hint:* Treat like base-26 conversion.
- [Count Primes](#) – Tags: Math, Sieve (easy). Count primes $< n$. *Hint:* Use the Sieve of Eratosthenes for efficiency.

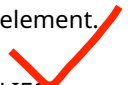
Sorting & Searching

Some easy problems involve simple sorting or binary search: **Sort Colors** (above), **Search Insert Position**, **Binary Search**, **Guess Number**, etc. Also include easy string/search puzzles like **First Bad Version** (binary search). These train binary search and sorting logic.

- [Binary Search](#) – *Tags:* Array, Binary Search. Standard binary search in a sorted array. *Hint:* While left $\leq$ right, compare mid to target and adjust.
 - [Guess Number Higher or Lower](#) – *Tags:* Binary Search. Guess number game implementation. *Hint:* Perform binary search guessing.
 - [First Bad Version](#) – *Tags:* Binary Search. Find first bad version via API. *Hint:* Binary search on versions 1..n.
 - [Search in Rotated Sorted Array](#) – *Tags:* Array, Binary Search (Medium). Find target in rotated sorted array. *Hint:* Modified binary search: check sorted half to decide search direction.
 - [Intersection of Two Linked Lists](#) – *Tags:* Array (logic), Pointers. (Already listed under Linked Lists).
 - [Height-Balanced BST - Preorder](#) – *Tags:* Recursion, Trees. (Already listed above.)
 - [Sort Array By Parity](#) – *Tags:* Array, Two Pointers. Move even numbers first. *Hint:* Use two pointers and swap.
 - [Squares of a Sorted Array](#) – *Tags:* Array, Two Pointers. Return squares of sorted array. *Hint:* Two pointers from ends to place largest square from right to left.
 - [Valid Sudoku](#) – *Tags:* Array, Hash Table. (Medium difficulty) Check row/col/3x3 validity. *Hint:* Use 3 hash sets for each group. (If needed for completeness.)
 - [Set Matrix Zeroes](#) – *Tags:* Array, In-place. Set row/col to zero if a zero appears. *Hint:* Use first row/column as markers.
- 

Stacks & Queues

Classic stack/queue problems include **Valid Parentheses** (above) and design problems. Other easy ones: **Min Stack**, **Implement Queue using Stacks**, etc. These test stack logic and simple design.

- [Min Stack](#) – *Tags:* Design, Stack. Implement a stack that supports retrieving the minimum element. *Hint:* Use an auxiliary stack to track mins or store pairs.
 - [Implement Queue using Stacks](#) – *Tags:* Design, Stack. Implement a FIFO queue with two LIFO stacks. *Hint:* Use one stack for enqueue, another for dequeue, transferring elements when needed.
 - [Implement Stack using Queues](#) – *Tags:* Design, Queue. (Optional) Use two queues to simulate a stack.
 - [Valid Parentheses](#) – (Already listed under Strings.)
 - [Sliding Window Maximum](#) – *Tags:* Heap/Deque (Hard). (Optional) Use deque for O(n).
 - [Queue Reconstruction by Height](#) – *Tags:* Array (Medium). (Often known trick problem.)
 - [Stock Spanner](#) – *Tags:* Stack, Design. (Not easy; skip.)
- 

Bits & Others

Some remaining classics involve bitwise operations or math puzzles. Good easy bit problems include **Hamming Distance**, **Number of 1 Bits**, **Reverse Bits**. Miscellaneous others: **Happy Number** (above), **Power of Two/Three** (above).

- [Hamming Distance](#) – *Tags*: Bit Manipulation. Compute number of differing bits between two ints. *Hint*: XOR the numbers and count set bits in result.
- [Number of 1 Bits](#) – *Tags*: Bit Manipulation. Count set bits in an integer. *Hint*: Repeatedly do `n &= (n-1)` to clear the lowest set bit and count.
- [Reverse Bits](#) – *Tags*: Bit Manipulation. Reverse bits of a 32-bit number. *Hint*: Loop bit-by-bit shifting into a new integer.
- [Add Digits](#) – *Tags*: Math. Repeatedly sum digits until one digit remains. *Hint*: Use digital root formula: return $1 + (n-1)\%9$ if $n>0$.
- [Ugly Number](#) – *Tags*: Math. Check if a number's prime factors are only 2,3,5. *Hint*: Repeatedly divide by 2,3,5 and see if result is 1.
- [Power of Two](#) – (Listed above.)
- [Power of Three](#) – (Listed above.)
- [Happy Number](#) – (Listed above.)
- [Fizz Buzz](#) – (Listed above.)

Study Tips and Practice Order

- **Practice by topic**: Work the list by topic (as above) to focus patterns (hashing, two pointers, recursion, etc.). Many of the above problems form a recommended 50–100 problem plan ¹⁰ ¹¹.
- **Daily targets**: Solve ~5 easy problems per day to finish 100 problems in ~20 days ¹². Focus on writing correct, clean code and understanding time/space complexity ² ¹².
- **Revision**: Revisit solved problems after a week to reinforce patterns ¹². Keep notes on solution techniques (e.g. “use hash map”, “two-pointer template”, “tree DFS”). Mix in old problems while practicing new ones.
- **Interview focus**: When practicing, time yourself (e.g. 30–40 mins) and simulate explaining your solution. Emphasize clarity: be ready to discuss complexity and why the approach works (important advice from experts ² ¹).
- **Further resources**: LeetCode's official lists (e.g. *Top 100 Liked* and *Top Interview Questions – Easy*), and many community guides echo these problems ⁴ ³. Use the above list as a checklist. Once comfortable, move on to medium problems.

By systematically solving these 100 easy problems (with full understanding of their patterns), you'll cover the foundational DSA topics frequently tested in coding interviews and placement rounds ¹ ³. Good luck!

Sources: Curated from LeetCode's recommended lists and community guides ¹ ³ ¹¹ (where we see recurring essentials like Two Sum, Valid Parentheses, etc.) and expert advice to master easy problems first ² ¹². Each problem above links to its official LeetCode page.

1 4 5 6 7 8 9 10 12 Top 50 Easy LeetCode Problems Every Beginner Must Solve | by Sowmiyan S | Medium

https://medium.com/@sowmiyan_s/top-50-easy-leetcode-problems-every-beginner-must-solve-eee50333c049

2 Don't overlook Leetcode's "easy" problems for interviews | Jim Lynch posted on the topic | LinkedIn

https://www.linkedin.com/posts/jimlynchcodes_the-last-word-on-length-of-last-word-activity-7365722750318964737-EDFi

3 Easy LeetCode Problems for Coding Interviews | Mohammad Azam Shaikh posted on the topic | LinkedIn

https://www.linkedin.com/posts/mohammad-azam-shaikh-8434a253_leetcode-codinginterview-dsa-activity-7349984070551945217-w2c0

11 (Locked Down Time) Productive way to use Leetcode at current situtation. - Discuss - LeetCode

<https://leetcode.com/discuss/post/600392/locked-down-time-productive-way-to-use-leetcode-at-current-situtation/>